

Tab 1

Getting an LLM to run natively on the Snapdragon 8 Elite's Hexagon HTP for Android requires cross-compiling with either the [Qualcomm Neural Processing (QNN) SDK](<https://www.qualcomm.com/developer/software/neural-processing-sdk-for-ai>) or Google's Edge LiteRT. Since open-source runtimes only support CPU execution natively, you must use proprietary toolchains to delegate the compute graph to the HTP. [1, 2, 3, 4]

Three main developer paths exist to target the Hexagon NPU for Android:

1. Google LiteRT with Qualcomm AOT Compilation (Highly Recommended) [5]

This is the most straightforward method for running models like Gemma 2 natively on the Snapdragon 8 Elite NPU. [3]

- * The Process: You must Ahead-of-Time (AOT) compile your model using the Google Edge LiteRT toolchain. This process transforms standard .tflite models into optimized NPU execution graphs. [3]

- * Setup Requirements: The compilation must be done on a Linux or WSL host because the toolchain is not available for native Windows/macOS Python. [3]

- * Actionable Tooling: Use the [LiteRT Gemma NPU Notebook](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md) as a guide to compile your model. [3]

2. Qualcomm GenieX SDK / QNN Runtime [4]

Qualcomm's official path relies on their QNN (Qualcomm Neural Processing) SDK, providing the best hardware-level access and power efficiency.

- * The Process: GenieX allows you to run models from Hugging Face or Qualcomm AI Hub directly on the Hexagon NPU using their C, Python, or Kotlin APIs.

- * Actionable Tooling: Check out the [Qualcomm GenieX Repository](<https://github.com/qualcomm/GenieX>) to use their open-source CLI and run LLMs on your target device. [6, 7]

3. Community Research Projects (e.g., Llama.cpp NPU Forks)

For open-source lovers trying to bend llama.cpp to run directly on the Hexagon HTP via Android Termux, the ecosystem relies on research branches. [8, 9, 10, 11]

- * The Process: You must cross-compile the codebase using the Android NDK and the Hexagon SDK to generate custom .so files (like libhttp_ops_skel.so) that bridge the open-source ggml backend to Qualcomm's FastRPC. [11, 12]

- * Actionable Tooling: Review the [Llama.cpp NPU Research Repo](<https://github.com/haozixu/llama.cpp-npu>) for an example of how to link custom HTP operator libraries to an LLM architecture. [8, 13]

Could you tell me what specific model size and architecture you are planning to deploy, and if you are building a native Android app vs. running a terminal environment like Termux? That will help determine the exact SDK configuration you need.

- [1] [\[https://github.com/ggml-org/llama.cpp/blob/master/docs/backend/snapdragon/windows.md\]](https://github.com/ggml-org/llama.cpp/blob/master/docs/backend/snapdragon/windows.md))
- [2] [\[https://mysupport.qualcomm.com/supportforums/s/question/0D5dK00000BcdbxSAB/can-snpe-sdk-run-llms-on-android-snapdragon-8-lite-if-not-which-sdk-can-be-used-please-provide-reference-materials\]](https://mysupport.qualcomm.com/supportforums/s/question/0D5dK00000BcdbxSAB/can-snpe-sdk-run-llms-on-android-snapdragon-8-lite-if-not-which-sdk-can-be-used-please-provide-reference-materials))
- [3] [\[https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md\]](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md))
- [4] [\[https://github.com/bpbonker/npurun\]](https://github.com/bpbonker/npurun))
- [5] [\[https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md\]](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md))
- [6] [\[https://github.com/qualcomm/GenieX\]](https://github.com/qualcomm/GenieX))
- [7] [\[https://github.com/qualcomm/GenieX\]](https://github.com/qualcomm/GenieX))
- [8] [\[https://github.com/haozixu/llama.cpp-npu\]](https://github.com/haozixu/llama.cpp-npu))
- [9] [\[https://www.reddit.com/r/termux/comments/1qswe99/llama_32_3b_on_snapdragon_8_elite_cpu_performance/\]](https://www.reddit.com/r/termux/comments/1qswe99/llama_32_3b_on_snapdragon_8_elite_cpu_performance/))
- [10] [\[https://medium.com/@mnp975/loading-models-on-gpu-534977993d68\]](https://medium.com/@mnp975/loading-models-on-gpu-534977993d68))
- [11] [\[https://github.com/haozixu/llama.cpp-npu\]](https://github.com/haozixu/llama.cpp-npu) ,)
- [12] [\[https://github.com/chraac/llama-cpp-qnn-builder\]](https://github.com/chraac/llama-cpp-qnn-builder))
- [13] [\[https://github.com/haozixu/llama.cpp-npu\]](https://github.com/haozixu/llama.cpp-npu)

Running Qwen 3.5 9B and Gemma 4 E4B IT natively on the Snapdragon 8 Elite's Hexagon Tensor Processor (HTP) requires adapting your precision approach. While the Snapdragon 8 Elite natively supports FP16 arithmetic on the HTP, deploying a 9B or even a 4B model at a pure FP16 weight precision will severely bottleneck memory bandwidth or cause the system to crash.

[1, 2, 3, 4]

An exact breakdown outlines how to target the HTP for these architectures:

⚠ Precision Strategy: Why Pure FP16 is Not Recommended

*

* The Memory Math: A pure FP16 model requires ~2 GB of RAM per 1 billion parameters.

* Gemma 4 E4B (4B) in FP16 would consume ~8 GB.

* Qwen 3.5 9B in FP16 would consume ~18 GB, exceeding typical Android system limitations for a single process. [3, 5, 6]

* The HTP Architecture: The Hexagon HTP achieves its hyper-efficient performance via Mixed Precision. The optimal approach is to use INT4 or INT8 quantized weights while keeping the key activation layers and mathematical operations in FP16. Pure unquantized floating-point inference is typically offloaded to the Adreno GPU. [1, 4, 7, 8, 9]

*

Step-by-Step Toolchains per Model [10] ## Path A: Deploying Qwen 3.5 9B (Via Qualcomm QNN / GenieX)

Because Qwen 3.5 utilizes standard transformer configurations with native rope/attention formats, it maps natively into Qualcomm's specialized LLM runtime. [2, 11, 12]

1. Quantization Setup: Download the official [Qualcomm AI Engine Direct (QNN) SDK](<https://docs.qualcomm.com/bundle/publicresource/topics/80-62010-1/qnn-workflow.html>) via the Qualcomm Developer Network.

2. Graph Conversion: Run the QNN model converter on your Linux host to transform the Qwen Hugging Face PyTorch weights into a QNN graph topology:

```
qnn-pytorch-converter -i /path/to/qwen3.5-9b -d input_ids 1x1 --input_type integer -o qwen3.5_http.json
```

[13]

3. Targeting HTP Mixed-Precision: Use the qnn-model-quantizer tool to enforce weight quantization while retaining FP16 precision for activations to maximize processing speed:

```
qnn-model-quantizer --input_network qwen3.5_http.json --output_precision htp_mixed_fp16_int4
```

[8, 13]

4. Integration: If you are building a terminal environment, utilize the open-source [Qualcomm GenieX CLI](<https://github.com/qualcomm/GenieX>) engine to load the resulting .bin and execute tokens directly on the device NPU. [2, 13]

Path B: Deploying Gemma 4 E4B IT (Via Google LiteRT) [14, 15]

Google's Gemma family has deep optimizations within Google's own [Edge LiteRT Toolchain](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md) (formerly TFLite).

1. Obtain the Baseline: Pull the official .tflite model format optimized for mobile from the Google AI Edge repository.

2. Ahead-of-Time (AOT) Compilation: To avoid fallback to the CPU, you must hard-bake the model operators for the HTP. Run the Google compiler on a Linux host, referencing the Snapdragon 8 Elite target system configuration:

```
# Targets the kHtpFp16 / mixed precision execution on the NPU backend
litert_compile --model_file=gemma4-e4b.tflite --output_file=gemma4_compiled_http.bin
--delegate=QNN --qnn_backend=HTP
```

3. Execution in Android: Load the compiled binary using the LiteRT C/C++ API in your application layer. [16]

Critical Verification Settings for Snapdragon 8 Elite

When running your execution layer inside an Android project or custom terminal pipeline, you must explicitly enable the underlying HTP precision flags in your backend options. If using a Java/Kotlin wrapper or C++ delegate, configure the execution provider options explicitly: [17]

```
// Explicitly enabling HTP FP16 internal precision mode for delegated  
mathTfLiteQnnDelegateHtpBackendOptions options;  
options.precision = TfLiteQnnDelegateHtpPrecision.kHtpFp16;
```

This flag guarantees that while the model weights remain highly compressed (saving massive battery and system RAM), the mathematical engine on the Snapdragon chip treats internal tensor calculations with full 16-bit floating-point precision to protect against quantization decay or reasoning looping errors. [3, 17, 18]

Are you planning to run these scripts on a Linux desktop host machine to compile the binaries, or are you trying to do the full compilation directly on the Android target device?

[1]

[<https://www.emergentmind.com>](<https://www.emergentmind.com/topics/qualcomm-sm8750-ab-snapdragon-8-elite-hexagon-tensor-processor-htp>)

[2]

[<https://www.mindstudio.ai>](<https://www.mindstudio.ai/blog/gemma-4-vs-qwen-3-5-open-weight-comparison>)

[3] [<https://www.youtube.com>](<https://www.youtube.com/watch?v=anhLHBi1pP4>)

[4]

[<https://www.lantronix.com>](<https://www.lantronix.com/products/snapdragon-8-elite-mobile-platform/>)

[5]

[<https://dev.to>](<https://dev.to/purpledoubled/how-to-run-googles-gemma-4-locally-with-ollama-all-4-model-sizes-compared-2pbh>)

[6]

[<https://www.reddit.com>](https://www.reddit.com/r/LocalLLaMA/comments/1sfr6u4/m5_max_128_gb_17_models_23_prompts_qwen_35_122b_is/)

[7]

[<https://docs.qualcomm.com>](<https://docs.qualcomm.com/doc/80-63195-1/topic/AI-hardware-cores-accelerators.html>)

[8]

[<https://docs.qualcomm.com>](<https://docs.qualcomm.com/doc/80-63442-10/topic/quantization.html>)

[9] [<https://arxiv.org>](<https://arxiv.org/html/2511.18674>)

[10]

<https://www.reddit.com>](https://www.reddit.com/r/LocalLLaMA/comments/1sg621w/benchmarked_gemma_4_e2b_vs_qwen_35_2b_on_a/)

[11]

<https://pytorch.org>](<https://pytorch.org/blog/up-to-580tps-new-speed-record-of-qwen3-5-397b-a-17b-on-gpu-for-agentic-workloads-with-tokenspeed/>)

[12]

<https://venturebeat.com>](<https://venturebeat.com/technology/alibabas-new-open-source-qwen3-5-medium-models-offer-sonnet-4-5-performance>)

[13]

<https://docs.qualcomm.com>](<https://docs.qualcomm.com/bundle/publicresource/topics/80-62010-1/qnn-workflow.html>)

[14] <https://huggingface.co>](<https://huggingface.co/SeatownSin/gemma-4-E4B-mtp-drafter>)

[15]

<https://www.reddit.com>](https://www.reddit.com/r/LocalLLaMA/comments/1tuygn6/using_gemma_4_e4b_with_the_litert_engine_24x/)

[16]

<https://www.reddit.com>](https://www.reddit.com/r/LocalLLaMA/comments/1sapl6k/my_first_impression_after_testing_gemma_4_against/)

[17]

<https://docs.qualcomm.com>](<https://docs.qualcomm.com/doc/80-63442-10/topic/support.html>)

[18] <https://www.youtube.com>](https://www.youtube.com/watch?v=vcbuN_d5qxc)

Official documentation and technical guides support the architectural setup for running your cloud-compiled models on the Snapdragon 8 Elite:

📖 Official Ecosystem Documentation

*

* Google AI Edge / LiteRT: Review the core [Qualcomm NPU (AI Engine Direct) with LiteRT Integration Documentation](<https://developers.google.com/edge/litert/next/qualcomm>). Google outlines how its new unified machine learning layout achieves a massive speedup over traditional mobile CPUs by linking directly to Qualcomm's silicon. [1, 2]

* Qualcomm AI Runtime (QAIRT) / QNN: The full deployment roadmap can be found in the [Qualcomm AI Engine Direct SDK Documentation Overview](https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-10/QNN_general_overview.html?product=1601111740009302). If you need specific environment configs for the runtime binaries, read the official [Qualcomm AI Runtime SDK Linux Setup Guide](https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-10/linux_setup.html) or explore the broader portal via the [Qualcomm Neural Processing SDK landing page] <https://www.qualcomm.com/developer/software/neural-processing-sdk-for-ai>

[3, 4, 5]

* The Hardware Layer: To deeply optimize how your app interacts with the chip's physical compute sections, study the [Qualcomm AI Engine Direct SDK HTP Backend

Guides](https://docs.qualcomm.com/nav/home/http_http.html?product=1601111740009302), which explicitly covers operations targeting the Hexagon Tensor Processor. [6, 7]

*

Execution & Deployment Blueprints

*

* For Gemma 4 E4B IT (via LiteRT): Use the official [Google LiteRT-Samples Gemma NPU Compilation and Push Blueprint](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md). Step 3 explicitly highlights how to push the model file layout containing the TF_LITE_AUX compiled payload over ADB directly onto your target phone without forcing CPU fallback. [8]

* For Qwen 3.5 9B (via QNN): If you are packaging the native libraries manually to run on a Linux-based or terminal Android subsystem, reference the [Qualcomm Innovation Development Kit (QIDK) GitHub Repository](<https://github.com/qualcomm/qidk>) for direct deployment scripts, and follow the device execution blocks outlined in the [Qualcomm Model Run and Verification Guide](<https://docs.qualcomm.com/doc/80-70014-15B/topic/qnn-run-model.html>). [9, 10]

* For Platform Runtimes: Read the official benchmark analysis and integration workflows on the [Google Developers Blog on Qualcomm NPU LiteRT Acceleration](<https://developers.googleblog.com/unlocking-peak-performance-on-qualcomm-npu-with-litert/>) to see how the Snapdragon 8 Elite processes models under 5ms. [1]

*

Are you aiming to wrap these cloud-compiled execution steps inside a native Android Studio Project using Java/Kotlin, or are you planning to run the binaries directly through an on-device command-line interface?

[1] [<https://developers.googleblog.com>](<https://developers.googleblog.com/unlocking-peak-performance-on-qualcomm-npu-with-litert/>)

[2] [<https://developers.google.com>](<https://developers.google.com/edge/litert/next/qualcomm>)

[3] [<https://docs.qualcomm.com>](https://docs.qualcomm.com/bundle/publicresource/topics/80-6344-2-10/QNN_general_overview.html?product=1601111740009302)

[4] [<https://docs.qualcomm.com>](https://docs.qualcomm.com/bundle/publicresource/topics/80-6344-2-10/linux_setup.html)

[5] [<https://www.qualcomm.com>](<https://www.qualcomm.com/developer/software/neural-processing-sdk-for-ai>)

[6]

[\[https://www.qualcomm.com/developer/software/qualcomm-ai-engine-direct-sdk \]](https://www.qualcomm.com/developer/software/qualcomm-ai-engine-direct-sdk)

[7]

[\[https://docs.qualcomm.com/nav/home/http_http.html?product=160111740009302 \]](https://docs.qualcomm.com/nav/home/http_http.html?product=160111740009302)

[8]

[\[https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md \]](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/qualcomm/llm_chatbot_npu/NPU_COMPILATION_GUIDE.md)

[9] [\[https://github.com/qualcomm/qidk \]](https://github.com/qualcomm/qidk)

[10]

[\[https://docs.qualcomm.com/doc/80-70014-15B/topic/qnn-run-model.html\]](https://docs.qualcomm.com/doc/80-70014-15B/topic/qnn-run-model.html)

Tab 2

